

Autonomous Cyber SOC

AI-Powered Cyberattack Detection & Autonomous Response Platform — Complete Technical Reference Guide

1. Executive Summary & Project Overview

The **Autonomous Cyber Security Operations Center (SOC)** is an end-to-end, automated cybersecurity platform designed to emulate a modern enterprise security center. In traditional Security Operations Centers, security analysts are overwhelmed by thousands of daily log alerts (alert fatigue), leading to delayed response times and uncontained breaches. This project solves alert fatigue by building a 5-stage automated pipeline that ingests raw network telemetry, detects cyber threats using hybrid (Rule-Based + Machine Learning) techniques, executes immediate containment playbooks, generates plain-English incident summaries using Google Gemini AI, and streams live telemetry to an interactive web dashboard with Human-in-the-Loop controls.

Core Capability	Technical Solution	Business / SOC Impact
Log Simulation	Streams realistic SSH logins, DNS/HTTP flows, brute-force bursts, C2 beacons, data exfiltration.	Provides realistic security telemetry without requiring expensive lab hardware.
Hybrid Detection	Combines deterministic rule thresholds with Scikit-Learn Isolation Forest anomaly detection.	Catches known signature attacks instantly while detecting unknown zero-day behavior.
Automated Response	Automated playbooks (<code>`block_ip`</code> , <code>`isolate_device`</code> , <code>`quarantine_transfer`</code>).	Reduces Mean Time to Respond (MTTR) from hours to sub-seconds.
AI Incident Summaries	Google Gemini 1.5 Flash LLM generates executive-friendly 4-part summaries.	Translates complex JSON logs into plain English for non-technical leadership.
Live SOC Dashboard	FastAPI backend + Async WebSockets + HTML5/Tailwind responsive UI.	Real-time threat feed with analyst Approve/Override controls.

2. System Architecture & End-to-End Data Pipeline

The application follows a strictly modular architecture composed of 7 primary parts plus a complete test suite. The data flow operates in a continuous real-time loop:

HIGH-LEVEL DATA FLOW ARCHITECTURE

- [1. **Data Simulator**] (Generates network logs & attack streams)
 - Yields raw JSON event logs
- [2. **Detection Engine**] (Evaluates Rules & Isolation Forest ML Anomaly Detector)
 - Output: Structured Threat Alert Dict (ID, Type, Severity, Evidence)
- [3. **Response Engine**] (Executes Security Playbooks)
 - Output: Containment Action (`block\_ip`, `isolate\_device`, `quarantine\_transfer`)
- [4. **LLM Explainability**] (Calls Google Gemini LLM API / Fallback Summarizer)
 - Output: Plain-English executive summary
- [5. **Backend API Server**] (FastAPI + Asyncio Task Loop + WebSocket Manager)
 - Broadcasts alerts to clients over `ws://localhost:8000/ws/alerts`
- [6. **Frontend SOC Dashboard**] (HTML5, Tailwind CSS, Async JS Event Handlers)
 - Renders live threat stream, stats counters & Human Analyst Controls

3. Detailed File-by-File Breakdown & Code Explanation

Below is an exhaustive breakdown of every file in the project, detailing its purpose, code implementation, functions, and schemas.

3.1 Part 1 — Data Simulator (`part1\_data\_simulator.py`)

Purpose: Simulates live enterprise network traffic and intersperses realistic attack vectors for testing detection models.

Key Concepts & Attack Vectors Generated:

- **Normal Traffic:** Random network flow logs (HTTP GET/POST, SSH port 22, DNS queries, normal bytes sent/received).
- **Brute-Force Attack:** A burst of >5 failed login attempts targeted at administrative accounts within a 60-second window.
- **C2 Beaconing:** Periodic, rhythmic outbound pings from an internal endpoint to known malicious Command & Control IPs (e.g., 185.220.101.12).
- **Data Exfiltration:** Unusually large outbound file transfers (>50 MB) originating during off-peak odd hours (23:00 to 05:00).

Code Highlights & Non-Blocking Design:

The primary generator is `log_stream(stream_duration_sec, normal_interval, inject_attacks, sleep_between)`. We added a `sleep_between` parameter. When collecting normal traffic for ML model training, `sleep_between=False` allows instantaneous dataset generation. In async loops, delays are handled asynchronously via `await asyncio.sleep()` to prevent freezing the server thread.

3.2 Part 2 — Detection Engine (`part2\_detection\_engine.py`)

Purpose: Analyzes incoming JSON log entries and flags potential security threats using a hybrid detection approach.

Detection Mechanisms:

1. **Rule-Based Detector:** Deterministic stateful inspection tracking failed logins per IP, ping frequency, and exfiltration thresholds.
2. **ML Anomaly Detector (`MLAnomalyDetector` class):** Uses Scikit-Learn's **Isolation Forest** algorithm. Isolation Forest isolates anomalies by randomly selecting a feature and splitting the value. Anomalies require fewer splits to isolate, yielding shorter tree path lengths.

Core Functions:

- `_detect_brute_force(log_entry, state)`: Flags IP if failed attempts ≥ 5 in 60s.
- `_detect_beaconing(log_entry, state)`: Flags IP if outbound connection interval to suspicious external IPs is below threshold.
- `_detect_exfiltration(log_entry, state)`: Flags transfer if byte count > 5MB during odd hours (23:00 - 05:00).
- `detect_alert(log_entry, state, ml_detector)`: Orchestrates rule evaluations, falling back to ML anomaly scoring if rules do not

trigger.

3.3 Part 3 — Response Engine (`part3\_response\_engine.py`)

Purpose: Implements automated Incident Response (IR) playbooks to instantly neutralize detected threats.

Playbook Mapping & State Management:

Alert Type	Automated Action	Target Resource	Action Description
brute_force	block_ip	Attacker Source IP	Applies firewall block rule to prevent further authentication attempts.
c2_beacon	isolate_device	Compromised Internal IP	Disables network interface to halt command & control communication.
data_exfil	quarantine_transfer	Session / Destination IP	Terminates active session and quarantines transferred files for forensic review.
anomaly	flag_for_review	Source IP / Account	Enriches log context and routes to Tier-2 analyst queue.

Idempotency Guard: Tracks blocked_ips and isolated_devices in a response_state set. If an IP is already blocked, subsequent triggers mark the status as skipped (already blocked), preventing duplicate execution.

3.4 Part 4 — LLM Explainability Layer (`part4\_llm\_explain.py`)

Purpose: Uses Large Language Models (Google Gemini API) to translate raw alert JSON and technical playbook responses into concise, plain-English executive summaries.

Prompt Engineering & Required Structure:

The system prompt instructs Gemini to output a structured summary covering 4 essential points:

- 1. **WHAT HAPPENED:** Concise plain-English threat description.
- 2. **HOW IT WAS DETECTED:** Rule or ML signal that flagged the anomaly.
- 3. **WHAT ACTION WAS TAKEN:** Automated response executed by the playbook.
- 4. **RECOMMENDED NEXT STEP:** Actionable guidance for human SOC analysts.

Resilient Fallback Mechanism:

If the GEMINI_API_KEY is not provided, or if the API call experiences network/quota limits, the system catches the exception and gracefully generates a high-quality fallback template using string interpolation. This guarantees zero server downtime.

3.5 Part 5 — Backend API Gateway (`part5\_api.py`)

Purpose: FastAPI application serving as the central nervous system. Wires Parts 1-4 together, provides REST endpoints, handles WebSocket connections, and serves the frontend dashboard.

Key Endpoints & WebSockets:

HTTP Method / Type	Endpoint Path	Function & Description
GET	/ & /dashboard	Serves the single-page HTML5 SOC Dashboard interface.
WebSocket	/ws/alerts	Real-time bi-directional socket streaming new alerts to browser clients.
GET	/api/alerts	Returns historical alerts with optional severity/type filtering.
POST	/api/alerts/{id}/approve	Analyst HITL endpoint: approves or overrides/reverts automated actions.
POST	/api/simulator/start	Launches non-blocking background simulation loop for N seconds.
POST	/api/simulator/stop	Stops the active background simulation task immediately.
GET	/api/simulator/status	Returns current simulation status (running: true/false).

3.6 Part 6 — Frontend SOC Dashboard (`part6\_dashboard.py` / `index.html`)

Purpose: Provides a dark-themed, glassmorphic Security Operations Center user interface built with HTML5, Tailwind CSS, and Vanilla JavaScript.

Key UI Features & Dynamic Behavior:

- **Live Telemetry Stream:** Establishes an automatic WebSocket connection (`ws://localhost:8000/ws/alerts`). As new threats are detected, alert cards animate into the feed with severity badges (Critical: Red, High: Orange, Medium: Yellow, Low: Green).
- **Threat Stats Counters:** Real-time metric boxes tracking Total Threat Alerts, High Severity, Critical Threats, and Brute-Force Attacks.
- **Interactive Incident Detail Panel:** Clicking any alert displays full metadata, raw evidence, automated response status, and Gemini AI plain-English summary.
- **Human Analyst Controls (HITL):** Provides **Approve Action** and **Override & Revert** buttons. When an analyst overrides an action, the API updates the status (e.g. overridden_by_analyst - IP unblocked) and broadcasts the updated state over WebSockets.
- **Top-Bar Simulator Controls:** Includes **Simulate Attack** and **Stop Simulation** buttons with live connection status indicators (■ SOC Live vs ■ Disconnected).

3.7 Part 7 & Test Suite (`part7\_demo\_script.py`, `test\_api.py`, `test\_pipeline.py`)

- **`part7\_demo\_script.py`:** A structured 3-minute presentation script designed for hackathon judges and executive demonstrations, outlining step-by-step actions for brute-force attacks, C2 beaconing, and analyst overrides.
- **`test\_pipeline.py`:** End-to-end integration test validating log generation -> rule/ML detection -> playbook execution -> summary creation without spinning up HTTP servers.
- **`test\_api.py`:** Automated API unit test suite using FastAPI's TestClient. Validates REST endpoints, JSON models, approval endpoints, and simulator control routes.

4. How the Website Works & Execution Workflow

To run and test the complete application manually, follow these 4 simple steps:

1. Start the Application Server:

```
python part5_api.py
```

This starts the FastAPI server listening on `http://localhost:8000`.

2. Open the SOC Dashboard:

Open your browser and navigate to **`http://localhost:8000/`**. The status badge will show ■ SOC Live.

3. Simulate Cyberattacks:

Click the green '**Simulate Attack**' button in the top right. The simulator starts feeding attack logs into the pipeline, generating alerts live on the UI.

4. Interact as a SOC Analyst:

Click any threat entry in the live stream. Inspect the AI-generated incident summary and click '**Approve Action**' or '**Override & Revert**' to test analyst control.

5. Top Interview Questions & Detailed Answers

Here are 15 comprehensive technical and behavioral interview questions related to this project, complete with model answers.

Q1: What is the main problem this Autonomous Cyber SOC project solves?

Ans: It addresses 'Alert Fatigue' in Security Operations Centers. Traditional SOC analysts are inundated with thousands of raw logs daily, leading to missed threats and long Mean Time to Respond (MTTR). This project automates log ingestion, hybrid anomaly detection, playbook containment, and AI explanation, reducing MTTR to sub-seconds while keeping a human analyst in the loop.

Q2: Why did you choose a Hybrid Detection model (Rules + Machine Learning)?

Ans: Deterministic rules provide 100% precision for known attack signatures (e.g., >5 failed logins = brute force) with near-zero latency. However, rules cannot catch novel zero-day attacks. The ML Anomaly Detector (Isolation Forest) models baseline normal traffic behavior and flags unexpected statistical deviations, providing defense-in-depth.

Q3: How does the Isolation Forest algorithm work in your project?

Ans: Isolation Forest isolates anomalies by randomly selecting a feature and split value. Because anomalies (like massive off-hours data exfiltration) are rare and statistically distinct, they are isolated near the root of decision trees, resulting in shorter path lengths. We extract numerical features (bytes sent, bytes received, port numbers, hour of day) and fit the model on normal baseline traffic.

Q4: Explain how WebSockets are used for real-time alerting.

Ans: Traditional HTTP polling creates unnecessary server load and latency. We implemented an async WebSocket endpoint (`/ws/alerts`) in FastAPI. When the backend pipeline detects an attack, `broadcast_alert()` serializes the alert to JSON and pushes it to all connected WebSocket browser clients in real time (<10ms latency).

Q5: How did you solve Python asyncio event loop blocking during log simulation?

Ans: Standard `time.sleep()` blocks Python's single-threaded event loop, preventing FastAPI from processing HTTP/WebSocket requests. We solved this by creating a non-blocking generator parameter `sleep_between=False` for fast execution, and using `await asyncio.sleep()` inside background tasks to yield execution back to the event loop.

Q6: What is Human-in-the-Loop (HITL) and why is it essential in AI Security?

Ans: Fully autonomous systems risk causing self-inflicted outages if an AI falsely blocks a legitimate mission-critical IP (False Positive). HITL ensures that while automated playbooks take immediate temporary containment actions, a human analyst retains ultimate authority to review AI summaries and approve or override/revert the action.

Q7: How does your LLM Explainability Layer handle Gemini API failure or quota limits?

Ans: We implemented a resilient fallback mechanism. The `generate_incident_summary()` function wraps the Gemini API call in a `try-except` block. If `GEMINI_API_KEY` is missing or the network request fails, it automatically returns a structured plain-English fallback summary generated from local alert metadata, ensuring 100% uptime.

Q8: What is Idempotency in Incident Response, and how is it enforced?

Ans: Idempotency means executing an operation multiple times yields the same result without unintended side effects. In `part3_response_engine.py`, we maintain global sets (`blocked_ips`, `isolated_devices`). If an attacker IP is already blocked, subsequent triggers check this state and mark the action as `skipped (already blocked)`, preventing duplicate firewall commands.

Q9: How did you fix CORS issues when connecting the frontend to the backend?

Ans: Browsers enforce Same-Origin Policy for HTTP and WebSockets. In FastAPI, we configured `CORSMiddleware` with `allow_origin_regex='.*'` and `allow_credentials=True`. This allows the dashboard to communicate seamlessly whether served via `localhost:8000`, `127.0.0.1`, or standalone HTML files.

Q10: Why was the `websockets` Python library required for Uvicorn?

Ans: Uvicorn is a lightweight ASGI web server implementation. While it handles standard HTTP/1.1 out-of-the-box, WebSocket protocol upgrades (`HTTP/1.1 101 Switching Protocols`) require an underlying WebSocket protocol engine like `websockets` or `wsproto`. Installing `websockets` enabled Uvicorn to accept WebSocket handshakes.

Q11: Describe the structure of an Alert JSON payload in your pipeline.

Ans: An Alert payload contains `alert\_id` (deterministic SHA256 hash), `type` (e.g., `brute\_force`, `c2\_beacon`), `severity` (`critical`, `high`, `medium`, `low`), `source\_ip`, `timestamp`, `raw\_evidence` (failed attempt counts, byte sizes, user targets), `action\_taken` (action, target, status, timestamp), and `incident\_summary` (LLM string).

Q12: How would you scale this architecture to handle 100,000 logs/second in production?

Ans: I would introduce Apache Kafka / AWS Kinesis as a distributed event streaming buffer, decouple the detection workers into stateless microservices using Celery/Redis or Kubernetes pods, store persistent alert logs in Elasticsearch / TimescaleDB, and use Redis Pub/Sub to scale WebSockets across multiple API gateway nodes.

Q13: What prompt engineering techniques were used for the Gemini LLM?

Ans: System prompt instructions enforced a strict role ('SOC analyst explaining to an executive'), length constraint (3-4 sentences), and 4 mandatory sections: WHAT HAPPENED, HOW IT WAS DETECTED, WHAT ACTION WAS TAKEN, and RECOMMENDED NEXT STEP. JSON alert data was injected dynamically via string formatting.

Q14: How do you prevent SQL Injection / Command Injection in automated playbook execution?

Ans: All IP targets and parameters are validated against Pydantic type schemas (`source\_ip: str` matching IPv4 regex patterns) and parameterized execution handlers rather than passing raw un-sanitized strings directly to OS shell execution.

Q15: What unit testing strategy did you implement for FastAPI?

Ans: We used `fastapi.testclient.TestClient` in `test\_api.py`. The test suite verifies endpoint status codes, JSON response schema compliance, simulator start/stop lifecycle management, and analyst approval/override decision recording.